ポリシー準拠墨塗り署名の構成と来歴管理への応用

梶田 海成[1,2,a)]　大竹 剛[1]

概要：生成 AI や高度な編集ツールの普及に伴い，メディアコンテンツの作成元と編集履歴を記録・検証するコンテンツ来歴技術の重要性が高まっている．C2PA などの既存フレームワークは，作成や編集に関する来歴情報をデジタル署名付きメタデータとして記録する一方，各編集が作成者の定めたポリシーに準拠することを，編集前情報を開示せずに暗号学的に検証する機構を提供していない．本稿では，秘匿された編集前データから公開された編集後データへの編集が，作成者指定のポリシーを満たすことを証明するポリシー準拠非対話ゼロ知識証明（PC-NIZK）を提案する．さらに，PC-NIZK を用いて，作成者による初期認証を不変のアンカーとして保持しつつ，複数の編集者による編集履歴を検証可能にする作成者アンカー型ポリシー準拠墨塗り署名（CAPC-SS）を提案する．CAPC-SS には，現在のコンテンツが編集履歴の最終コミットメントルートに対応することを，コミットメントの開示情報を明らかにせずに証明するコンテンツ・ルート整合性非対話ゼロ知識証明（CR-NIZK）も組み込む．CAPC-SS について，作成者アンカー不変性，コンテンツ・ルート整合性，ポリシー準拠性，編集者認証性，および編集前情報の秘匿性を定式化し，提案する一般構成がこれらの性質を満たすことを示す．最後に，CAPC-SS を C2PA 型来歴管理へ適用することで，作成者が最初に認証したコンテンツから，作成者指定のポリシーに準拠する編集を経て現在のコンテンツが生成されたことを検証する機能を補完できることを示す．

キーワード：墨塗り署名，非対話ゼロ知識証明，ポリシー準拠，来歴情報，C2PA

Policy-Compliant Sanitizable Signatures and Their Application to Content Provenance

Kaisei Kajita[1,2,a)]　Go Ohtake[1]

Abstract: The growing use of generative AI and advanced editing tools has increased the importance of provenance technologies that record and verify the origins and edit histories of media content. Existing frameworks such as C2PA record provenance information about creation and editing as digitally signed metadata. However, they do not provide a cryptographic mechanism for verifying, without disclosing pre-edit information, that each edit complies with a policy specified by the creator. We first propose Policy-Compliant Non-Interactive Zero-Knowledge Proofs (PC-NIZK), which prove that an edit from hidden pre-edit data to public post-edit data satisfies a creator-specified policy. Building on PC-NIZK, we then propose Creator-Anchored Policy-Compliant Sanitizable Signatures (CAPC-SS), which retain the creator's initial authentication as an immutable anchor while enabling verification of edit histories involving multiple editors. CAPC-SS also incorporates a Content-Root Consistency NIZK proof (CR-NIZK), which proves that the current content corresponds to the final commitment root of the edit history without revealing the commitment openings. We formalize creator-anchor invariance, content-root consistency, policy compliance, editor authentication, and privacy of pre-edit information, and prove that our generic construction satisfies these properties. Finally, we show that CAPC-SS can complement C2PA-style provenance management with cryptographic verification that the current content was derived from content initially authenticated by the creator through edits complying with the creator-specified policy.

Keywords: sanitizable signature, non-interactive zero-knowledge proof, policy compliance, content provenance, C2PA

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

1.1 背景

生成 AI や高度な編集ツールの普及により，画像，映像，音声等のメディアコンテンツを容易に生成・加工できるようになり，メディアコンテンツの真正性が社会的課題となっている．そこで，コンテンツの作成元や編集履歴を署名付きメタデータとして記録・提示する来歴情報管理の重要性が高まっている．C2PA（Coalition for Content Provenance and Authenticity） [1], [2] は，作成元，編集アクション，使用ツール，素材等の真正性を検証するための実用的な来歴情報フレームワークであり，Adobe Photoshop [3], OpenAI の ChatGPT [4], Google Pixel 10 [5] 等，幅広い製品・サービスで導入が進んでいる．C2PA により，メディアコンテンツがどのような制作過程を経たかを確認でき，コンテンツの信頼性判断に資することが期待される．

C2PA の基本構成． C2PA の基本的な構成を図 1 に示す．アセットは検証対象となる画像や映像等のメディアコンテンツである．アサーションは，作成・編集アクション，使用機器，アセットの生成・編集に用いた素材等に関する個別の記述である．クレームは，アサーション群への参照とアセットとのバインディング[*1]に必要な情報を含む署名対象となる情報であり，マニフェストの署名者はこれにデジタル署名する．アサーション，クレーム，クレーム署名をまとめたものが C2PA マニフェストであり，複数のマニフェストの連鎖によって来歴が構成される [1]．なお，C2PA はコンテンツの制作過程を示す情報を提供するものであり，コンテンツに含まれる情報そのものの真偽を判定するものではない．

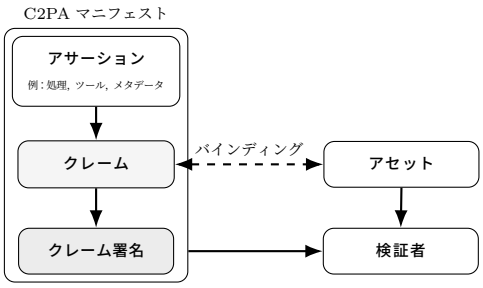


図 1 C2PA の基本構成．

複数の編集と作成者アンカー． メディアコンテンツは，例えば撮影，編集，配信，二次利用等の複数の処理を経て利用者へ届けられる．C2PA では，各処理段階でマニフェストが生成されるため，作成者が最初に生成したクレーム署名は来歴中に保持されるものの，現在のコンテンツに直接対応するのは，後続の処理段階で生成されたマニフェストである．この現在のコンテンツに対応する最新のマニフェストはアクティブマニフェストと呼ばれる [1]．

一方，報道・放送の現場では，複数の編集を経た後も，作成者が最初に認証したコンテンツを基準として現在のコンテンツを検証したい場合がある．例えば，事故現場を撮影した映像では，その後に編集が加えられた場合でも，一次情報の所有者が最初に認証したコンテンツに由来することを継続して確認できることが望ましい．そこで本稿では，後続の編集後も作成者による最初の認証を検証の基準として保持するための固定情報を作成者アンカーとして導入する．

ポリシー準拠墨塗り． 報道の現場では，前述した事故映像等の例において，映像に含まれる顔・車両番号・個人情報等を秘匿するため，コンテンツの一部の情報の削除・墨塗りを施す場合がある．C2PA では，墨塗りを行ったという処理自体を署名付きの来歴情報として改ざん検知可能な形で記録できる．しかし，「墨塗りを行った」という記録が正しいことと，その墨塗りが作成者の許可した範囲内であることは同一ではない．

本稿では，この問題を扱うため，コンテンツを編集単位となる複数のセグメントに分割して考える．例えば画像では，分割された領域や意味的な領域を一つのセグメントとして扱う．図 2 に本稿が対象とする問題設定を模式的に示す．一次情報の初期コンテンツを所有する作成者が顔や ID を含む特定のセグメントだけに墨塗りを許可し，検証者は，複数の編集を経た後も，許可されていないセグメントが変更されていないことを確認したい場合を考える．このとき，その編集が許可された条件を満たすことを確認するために，編集前値と編集後値との関係を検証する必要がある．しかし，編集前値を検証者へ開示すると，墨塗りによって情報を秘匿する目的が失われる．そこで，作成者アンカーを固定したまま，編集前情報を開示することなく，各編集が作成者の定めた許可条件を満たすことを検証できる仕組みが必要となる．

1.2 本稿の貢献

本稿では，まずセグメント単位の編集が作成者の定めた条件を満たすことを，編集前情報を開示せずに証明するポリシー準拠非対話ゼロ知識証明（PC-NIZK）を提案する．また，コンテンツとそのコミットメント列から得られる Merkle root との整合性を証明するコンテンツ・ルート整合性非対話ゼロ知識証明（CR-NIZK）を導入する．さらに，これらを用いて，作成者アンカーを保持したまま複数編集

[1] 日本放送協会 放送技術研究所
NHK Science & Technology Research Laboratories
[2] 東京大学
The University of Tokyo
[a)] kajita.k-bu@nhk.or.jp

[*1] バインディングには，暗号学的ハッシュによるハードバインディングや，フィンガープリント，電子透かしによるソフトバインディング等が用いられる [1].

© 2026 Information Processing Society of Japan

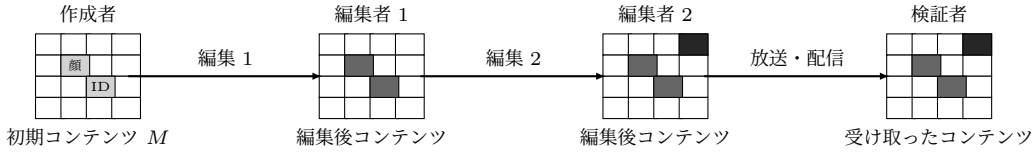


図 2 本稿が対象とするポリシー準拠墨塗りの問題設定．初期コンテンツの所有者である作成者が顔や ID などの特定のセグメントに対する墨塗りのみを許可した場合，検証者は編集前情報を知ることなく，複数の編集を経たコンテンツに許可外の変更が含まれていないことを検証する必要がある．ここでは顔や ID 以外のセグメントが墨塗りされている．

者による編集履歴を検証する作成者アンカー型ポリシー準拠墨塗り署名（CAPC-SS）を構成する．本稿の主な貢献は次の通りである．

（1） セグメント単位の編集に対する PC-NIZK を定式化し，完全性，知識抽出可能性，およびゼロ知識性を定義する．

（2） コンテンツと Merkle root との整合性を，コミットメントの開示情報を公開せずに証明する CR-NIZK を定義する．さらに，PC-NIZK，CR-NIZK，コミットメント，Merkle 木，およびデジタル署名を用いた CAPC-SS の構文と一般構成を示す．

（3） CAPC-SS について，作成者アンカー不変性，コンテンツ・ルート整合性，編集者認証性，ポリシー準拠性，および編集前情報の秘匿性を形式化し，各要素技術の安全性仮定の下で一般構成がこれらを満たすことを示す．

（4） CAPC-SS を C2PA 型来歴管理へ適用し，C2PA によるアセットと来歴情報との対応検証を補完して，現在コンテンツと CAPC-SS の現在ルートとの整合性，および各編集のポリシー準拠性を検証する方法を示す．

1.3 関連研究

墨塗り署名は，作成者が許可した部分を編集者が変更した後も，変更後文書の真正性を検証可能とするデジタル署名である [6]．その後，説明責任性，透明性，非連結性，プライバシー保護等が研究されてきた [7], [8], [9], [10], [11]．ポリシーベース墨塗り署名では属性に基づいて編集者の権限を制御する [12], [13]．また，γ-墨塗り署名は変更可能なブロック数を制限する [14]．

Nandakumar らは，セグメントごとのカメレオンハッシュを Merkle 木に集約し，墨塗り署名とブロックチェーンを用いて変更履歴を管理する方式を提案した [15]．

PhotoProof [16] は，署名済みの初期画像から許可された画像変換だけを経て現在の画像が生成されたことを Proof-Carrying Data [17] により証明し，切り抜き等で除去された情報もゼロ知識で秘匿できる．VerITAS [18] は，C2PA における編集アプリケーションへの信頼が必要であることを課題として，署名済み画像に許可された編集だけが適用されたことを zk-SNARK で証明する方式を提案した．特に，従来方式では困難であった 30 メガピクセル級の画像に対しても実用的な証明生成を可能とした．PhotoProof および VerITAS は，編集前画像を秘匿したまま許可された画像変換を証明する点で本研究に近い．一方，これらが画像変換の正当性の証明を主眼とするのに対し，本稿では，作成者による最初の認証を固定したまま，各編集について編集者を認証し，作成者の定めた許可条件への適合性を検証する．

2. システムモデル

本章では本稿で扱うシステムモデルとポリシーおよびコミットメントによる Merkle ツリーを導入する．

2.1 モデル

編集前コンテンツをセグメント列 $M = (m_1, \ldots, m_N)$ とし，編集対象のインデックスを $i \in \{1, \ldots, N\}$ とする．セグメント単位の編集後のコンテンツを $M' = (m_1, \ldots, m_{i-1}, m'_i, m_{i+1}, \ldots, m_N)$ とする．すなわち，各編集ステップでは局所セグメント i だけを m_i から m'_i へ変更する．編集者識別子を $j \in \{1, ..., n\}$，編集者公開鍵集合を $\mathcal{PK} = \{(1, pk_1), \ldots, (n, pk_n)\}$ とする．

2.2 ポリシー

ポリシーを決定的多項式時間述語 $\mathsf{Pol}(j, i, m_i, m'_i) \in \{0, 1\}$ で定義し，$\mathsf{Pol}(j, i, m_i, m'_i) = 1$ ならば，編集者 j がセグメント i を m_i から m'_i へ変更することが許可されるものとする．例えば，特定の編集者だけが指定セグメントを固定の墨塗り値へ置換する条件，数値を許可範囲内へ丸める条件，指定メタデータを削除する条件等を表現できる．PC-NIZK は，編集前値とコミットメントの開示情報，およびポリシーの充足を，m_i を秘匿したまま証明する．編集者の認証は後述の編集者署名が担い，PC-NIZK は局所的なポリシー準拠性だけを証明する．

2.3 コミットメントと Merkle ツリー

コミットメント． コミットメント方式 [19] は，値を秘匿したまま固定し，後に開示できる暗号プリミティブである．$\rho \in \mathcal{R}_{\mathrm{com}}$ を内部乱数としたとき，値 $m \in \mathcal{M}$ に対するコミットメント方式 $\mathsf{Com} : \mathcal{M} \times \mathcal{R}_{\mathrm{com}} \to \mathcal{C}$ を $C = \mathsf{Com}(m; \rho)$

© 2026 Information Processing Society of Japan

とする．Com には秘匿性と束縛性を要求する．直感的には，秘匿性はコミットメントから m を識別できない性質であり，束縛性は同一のコミットメントを異なる値に開示できない性質である．

Merkle ツリー． コンテンツ $M = (m_1, \ldots, m_N) \in \mathcal{M}$ の各セグメント m_i に対して $C_i = \mathsf{Com}(m_i; \rho_i)$ を計算し，コミットメント列 $\mathbf{C} = (C_1, \ldots, C_N)$ を Merkle 木の葉ノード [20] とする．暗号学的ハッシュ関数 H を用いて得られる Merkle 木のルートノードを $r = \mathsf{MerkleRoot}_H(C_1, \ldots, C_N)$ とする．

ここで，編集者 j がセグメント i を m_i から m_i' へ変更したときの編集前後の各コミットメントを

$$C_i = \mathsf{Com}(m_i; \rho_i), \qquad C_i' = \mathsf{Com}(m_i'; \rho_i')$$

とする．図 3 に示すように，編集前後の Merkle root をそれぞれ r_{in}，r_{out} とすると，

$$r_{\mathrm{in}} = \mathsf{MerkleRoot}_H(C_1, \ldots, C_i, \ldots, C_N),$$
$$r_{\mathrm{out}} = \mathsf{MerkleRoot}_H(C_1, \ldots, C_i', \ldots, C_N)$$

となる．また，編集前の C_i に対する Merkle 認証パスを $\mu_i \leftarrow \mathsf{MerklePath}_H(i, C_1, \ldots, C_N)$ とする．検証者は，C_i が編集前の Merkle 木の i 番目の葉であることを確認するための関数 $\mathsf{VerifyPath}_{\mathsf{H}}(r_{\mathrm{in}}, i, C_i, \mu_i) = 1$ を計算する．また，C_i を C_i' へ置き換えて Merkle root を再計算する関数 $\mathsf{UpdateMerkle}_H(r_{\mathrm{in}}, i, C_i, C_i', \mu_i) = r_{\mathrm{out}}$ を用いて，編集後のルートノード r_{out} を確認する．

3. PC-NIZK

本章では，まずポリシー準拠関係を導入し，セグメント単位の編集に対するポリシー準拠非対話ゼロ知識証明 PC-NIZK を定義する．PC-NIZK により，検証者は編集前値を知ることなく，編集が作成者の定めたポリシーを満たすことを確認できる．

3.1 ポリシー準拠関係

コンテンツを識別する値を sid，公開パラメータの識別子を ppid とする．PC-NIZK の公開ステートメント x_i と秘密のウィットネス w_i を，それぞれ

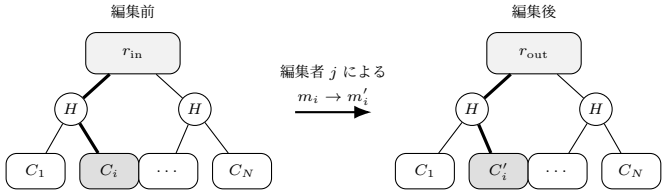


図 3 セグメント単位のコミットメントと Merkle root の更新．編集対象セグメント i のコミットメントを C_i から C_i' へ更新すると，Merkle root は r_{in} から r_{out} へ更新される．太線は対象セグメントから root までの Merkle 認証パスを表す．

$$x_i = (\mathsf{sid}, j, i, C_i, C_i', m_i', \mathsf{Pol}, r_{\mathrm{in}}, r_{\mathrm{out}}, \mathsf{ppid}),$$
$$w_i = (m_i, \rho_i, \rho_i')$$

と定義する．ここで，m_i は秘匿される編集前値，m_i' は公開される編集後値であり，C_i, C_i' はそれぞれに対するコミットメントである．コミットメント方式 Com に対するポリシー準拠関係 $\mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}}$ を

$$\mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}} = \left\{ (x_i, w_i) : \begin{array}{l} C_i = \mathsf{Com}(m_i; \rho_i), \\ C_i' = \mathsf{Com}(m_i'; \rho_i'), \\ \mathsf{Pol}(j, i, m_i, m_i') = 1 \end{array} \right\}$$

と定義する．すなわち，$(x_i, w_i) \in \mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}}$ であることは，次の三つが同時に成立することを意味する．(i) C_i が非公開の編集前値 m_i に対する正しいコミットメントであること，(ii) C_i' が公開された編集後値 m_i' に対する正しいコミットメントであること，および (iii) m_i から m_i' への編集が作成者の定めたポリシー Pol を満たすことである．なお，対象セグメントに対する Merkle 認証パスの正当性と，r_{in} から r_{out} へのルート更新については，通常のハッシュ検証によって確認するため，$\mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}}$ の判定条件には含めない．一方，sid，r_{in}，r_{out}，および ppid は履歴検証に必要な文脈情報として公開ステートメントに含める．

3.2 構文

PC-NIZK は次の 3 つの確率的多項式時間 (PPT) アルゴリズムからなる．

- $pp_{\mathrm{pc}} \leftarrow \mathsf{PCNIZK.Setup}(1^\lambda)$：セキュリティパラメータを入力し，公開パラメータを出力する．
- $\pi_i \leftarrow \mathsf{PCNIZK.Prove}(pp_{\mathrm{pc}}, x_i, w_i)$：公開ステートメント x_i とウィットネス w_i から証明 π_i を生成する．
- $b \leftarrow \mathsf{PCNIZK.Verify}(pp_{\mathrm{pc}}, x_i, \pi_i)$：証明を検証し，$b \in \{0, 1\}$ を出力する．

3.3 安全性

PC-NIZK には，ポリシー準拠関係 $\mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}}$ に対する完全性，知識抽出可能性，およびゼロ知識性を要求する．

定義 1 (完全性)．任意の $(x, w) \in \mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}}$ について，$pp_{\mathrm{pc}} \leftarrow \mathsf{PCNIZK.Setup}(1^\lambda)$, $\pi \leftarrow \mathsf{PCNIZK.Prove}(pp_{\mathrm{pc}}, x, w)$ としたとき，

$$\Pr[\mathsf{PCNIZK.Verify}(pp_{\mathrm{pc}}, x, \pi) = 1] \geq 1 - \mathsf{negl}(\lambda)$$

が成り立つとき，*PC-NIZK* は完全性を満たすという．

定義 2 (知識抽出可能性)．*PC-NIZK* について，*PPT* アルゴリズム ExtSetup および Ext が存在するとする．ExtSetup は $(pp_{\mathrm{pc}}, \tau_{\mathsf{ext}}) \leftarrow \mathsf{ExtSetup}(1^\lambda)$ を出力し，pp_{pc} の分布は $\mathsf{PCNIZK.Setup}(1^\lambda)$ の出力と計算量的に識別不能であるとする．

© 2026 Information Processing Society of Japan

任意の PPT 攻撃者 $\mathcal{A}$ に対して，次の試行を定義する．

$\underline{\mathsf{Exp}^{\mathsf{ke}}_{\mathsf{PC\text{-}NIZK},\mathcal{A}}(1^\lambda):}$
1. $(pp_{\mathsf{pc}}, \tau_{\mathsf{ext}}) \leftarrow \mathsf{ExtSetup}(1^\lambda)$.
2. $(x^*, \pi^*) \leftarrow \mathcal{A}(pp_{\mathsf{pc}})$.
3. *If* $\mathsf{PCNIZK.Verify}(pp_{\mathsf{pc}}, x^*, \pi^*) \neq 1$, *return* 0.
4. $w^* \leftarrow \mathsf{Ext}(\tau_{\mathsf{ext}}, x^*, \pi^*)$.
5. *Return* 1 *iff* $(x^*, w^*) \notin \mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}}$.

任意の PPT 攻撃者 $\mathcal{A}$ について

$$\Pr\left[\mathsf{Exp}^{\mathsf{ke}}_{\mathsf{PC\text{-}NIZK},\mathcal{A}}(1^\lambda) = 1\right] \leq \mathsf{negl}(\lambda)$$

が成り立つとき，*PC-NIZK* は知識抽出可能性を満たすという．

定義 3 (ゼロ知識性)．*PC-NIZK* について，*PPT* アルゴリズム SimSetup および SimProve が存在するとする．SimSetup は $(pp_{\mathrm{pc}}, \tau_{\mathrm{sim}}) \leftarrow \mathsf{SimSetup}(1^\lambda)$ を出力し，pp_{pc} の分布は $\mathsf{PCNIZK.Setup}(1^\lambda)$ の出力と計算量的に識別不能であるとする．さらに，任意の $(x, w) \in \mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}}$ について，$\pi \leftarrow \mathsf{PCNIZK.Prove}(pp_{\mathrm{pc}}, x, w)$ から得られる実際の証明と，$\tilde{\pi} \leftarrow \mathsf{SimProve}(\tau_{\mathrm{sim}}, x)$ から得られるシミュレートされた証明が，任意の PPT 識別器に対して計算量的に識別不能であるとき，*PC-NIZK* はゼロ知識性を満たすという．

4. CR-NIZK

PC-NIZK は，各編集が作成者の定めたポリシーを満たすことを，編集前値を開示せずに証明する．一方，PC-NIZK と Merkle root の更新履歴だけでは，履歴の最終ルートが検証対象の現在コンテンツに対応することまでは確認できない．そこで本稿では，コンテンツとそのコミットメント列から得られる Merkle root との整合性を，コミットメントの開示情報を公開せずに証明するコンテンツ・ルート整合性非対話ゼロ知識証明（Content-Root Consistency NIZK; CR-NIZK）を導入する．

4.1 コンテンツ・ルート整合関係

コンテンツを $M = (m_1, \ldots, m_N)$，その各セグメントに対するコミットメント列から得られる Merkle root を r とする．CR-NIZK の公開ステートメントとウィットネスをそれぞれ $x_{\mathrm{cr}} = (M, r)$, $w_{\mathrm{cr}} = (\rho_1, \ldots, \rho_N)$ とする．コミットメント方式 Com に対するコンテンツ・ルート整合関係を

$$\mathcal{R}^{\mathsf{cr}}_{\mathsf{Com}} = \left\{(x_{\mathrm{cr}}, w_{\mathrm{cr}}) : r = \mathsf{Com}_H\left((\mathsf{Com}(m_i; \rho_i))_{i=1}^N\right)\right\}$$

と定義する．すなわち，$(x_{\mathrm{cr}}, w_{\mathrm{cr}}) \in \mathcal{R}^{\mathsf{cr}}_{\mathsf{Com}}$ であることは，公開されたコンテンツ M の各セグメントを w_{cr} に含まれる乱数でコミットしたとき，そのコミットメント列から計算される Merkle ルートが r に一致することを意味する．

4.2 構文

CR-NIZK は次の 3 つの PPT アルゴリズムからなる．

- $pp_{\mathrm{cr}} \leftarrow \mathsf{CRNIZK.Setup}(1^\lambda)$：セキュリティパラメータを入力し，公開パラメータを出力する．
- $\pi_{\mathrm{cr}} \leftarrow \mathsf{CRNIZK.Prove}(pp_{\mathrm{cr}}, x_{\mathrm{cr}}, w_{\mathrm{cr}})$：公開ステートメント x_{cr} とウィットネス w_{cr} から証明 π_{cr} を生成する．
- $b \leftarrow \mathsf{CRNIZK.Verify}(pp_{\mathrm{cr}}, x_{\mathrm{cr}}, \pi_{\mathrm{cr}})$：証明を検証し，$b \in \{0, 1\}$ を出力する．

4.3 安全性

CR-NIZK には，$\mathcal{R}^{\mathsf{cr}}_{\mathsf{Com}}$ に対する完全性，知識抽出可能性，およびゼロ知識性を要求する．これらは Section 3.3 の PC-NIZK に対する各定義において，PCNIZK を CRNIZK へ，pp_{pc} を pp_{cr} へ，$\mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}}$ を $\mathcal{R}^{\mathsf{cr}}_{\mathsf{Com}}$ へ置き換えて定義する．

完全性，知識抽出可能性，およびゼロ知識性により，それぞれ正しいコンテンツ・ルート整合性証明の受理，受理証明からのコミットメント乱数の抽出，および当該乱数の秘匿を保証する．

5. CAPC-SS

本章では，PC-NIZK と CR-NIZK を用いて，作成者による初期認証を固定したまま，複数の編集者による編集履歴を検証する CAPC-SS を構成する．

5.1 システムモデル

主体は，作成者 $\mathcal{C}$，編集者 $\mathcal{E}_j$，および検証者 $\mathcal{V}$ である．作成者は，初期コンテンツ M_0，ポリシー Pol，編集者公開鍵集合 $\mathcal{PK}$，およびセグメント化・正規化規則識別子 segid を設定する．編集者は，現在の各セグメントとコミットメントの開示情報からなる非公開状態 open を保持する．実際のコンテンツ編集は外部アプリケーションが実行し，Sanitize は，編集対象集合 I と編集後値集合 $\Delta = \{m'_i\}_{i \in I}$ を用いて認証情報と非公開状態を更新する．検証者は，作成者公開鍵 pk_C，現在コンテンツ M，および公開認証情報 Σ を用いて，作成者が認証した初期状態から M に至る編集履歴を検証する．

5.2 構文

CAPC-SS は，6 つの PPT アルゴリズム $\mathsf{CAPC\text{-}SS} = (\mathsf{Setup}, \mathsf{CKeyGen}, \mathsf{OKeyGen}, \mathsf{Sign}, \mathsf{Sanitize}, \mathsf{Verify})$ から構成される．

- $pp \leftarrow \mathsf{Setup}(1^\lambda, N)$: 公開パラメータ pp を生成する．
- $(sk_C, pk_C) \leftarrow \mathsf{CKeyGen}(pp)$: 作成者の署名鍵対を生成する．
- $(sk_j, pk_j) \leftarrow \mathsf{OKeyGen}(pp, j)$: 編集者 j の署名鍵対を生成する．

© 2026 Information Processing Society of Japan

- $(\Sigma_0, \mathsf{open}_0)/\perp \leftarrow \mathsf{Sign}(pp, sk_C, M_0, \mathsf{Pol}, \mathcal{PK}, \mathsf{segid})$: 初期認証情報と非公開状態を生成する．
- $(\Sigma', \mathsf{open}')/\perp \leftarrow \mathsf{Sanitize}(pp, sk_j, j, \Sigma, \mathsf{open}, I, \Delta)$: 外部アプリケーションによる編集結果に対応する認証情報と非公開状態を生成する．
- $b \leftarrow \mathsf{Verify}(pp, pk_C, M, \Sigma)$: 現在コンテンツと認証情報を検証し，$b \in \{0,1\}$ を出力する．

定義 4 (正当性)．正しく生成された公開パラメータと鍵の下で，Sign および各 Sanitize が $\perp$ を出力せず，すべての編集が Pol を満たすとする．このとき，最終コンテンツ M_T と認証情報 Σ_T について $\Pr[\mathsf{Verify}(pp, pk_C, M_T, \Sigma_T) = 1] \geq 1 - \mathsf{negl}(\lambda)$ が成り立つとき，*CAPC-SS* は正当性を満たすという．

5.3 一般構成

一般構成では，EUF-CMA 安全なデジタル署名方式 $\mathsf{Sig} = (\mathsf{Sig.KeyGen}, \mathsf{Sig.Sign}, \mathsf{Sig.Verify})$，コミットメント方式 Com，衝突困難なハッシュ関数 H，PC-NIZK，および CR-NIZK を用いる．Setup は各方式の公開パラメータを生成し，$pp = (pp_{\mathrm{pc}}, pp_{\mathrm{cr}}, pp_{\mathrm{com}}, H, N, \mathsf{ppid})$ を出力する．ここで，$\mathsf{ppid} = H(pp_{\mathrm{pc}} \| pp_{\mathrm{cr}} \| pp_{\mathrm{com}} \| N)$ である．CKeyGen と OKeyGen は Sig の鍵生成を実行する．

認証情報． 初期ルート r_0，セグメント数 N，およびコンテンツ識別子 sid から，作成者アンカーを $\mathsf{anc} = (r_0, N, \mathsf{sid})$ とする．σ_C を作成者署名とする．第 k 番目の履歴エントリを $\ell_k = (\widehat{\ell}_k, \sigma_{j_k})$ とし，

$$\widehat{\ell}_k = (j_k, i_k, C_k, C'_k, m'_k, \mu_k, r_{\mathrm{in},k}, r_{\mathrm{out},k}, \pi_k)$$

とする．ここで，μ_k は編集前コミットメント C_k に対する Merkle 認証パス，π_k は PC-NIZK 証明，σ_{j_k} は編集者署名である．履歴を $\mathsf{hist} = (\ell_1, \ldots, \ell_t)$ とし，現在ルートを r，現在コンテンツと r に対する CR-NIZK 証明を π_{cr} とする．公開認証情報を

$$\Sigma = (\mathsf{anc}, \sigma_C, \mathsf{Pol}, \mathcal{PK}, \mathsf{segid}, (r, t), \pi_{\mathrm{cr}}, \mathsf{hist})$$

とし，非公開状態を $\mathsf{open} = ((m_i, \rho_i))_{i=1}^N$ とする．

初期認証． Sign は，初期コンテンツ $M_0 = (m_1, \ldots, m_N)$ について $C_i = \mathsf{Com}(m_i; \rho_i)$ を計算し，$r_0 = \mathsf{MerkleRoot}_H(C_1, \ldots, C_N)$ とする．sid を生成して $\mathsf{anc} = (r_0, N, \mathsf{sid})$ とし，

$$\sigma_C \leftarrow \mathsf{Sig.Sign}(sk_C, \mathsf{anc} \| \mathsf{Pol} \| \mathcal{PK} \| \mathsf{segid} \| \mathsf{ppid})$$

を生成する．さらに，$x_{\mathrm{cr},0} = (M_0, r_0)$ に対する CR-NIZK 証明 $\pi_{\mathrm{cr},0}$ を生成し，

$$\Sigma_0 = (\mathsf{anc}, \sigma_C, \mathsf{Pol}, \mathcal{PK}, \mathsf{segid}, (r_0, 0), \pi_{\mathrm{cr},0}, \emptyset)$$

および $\mathsf{open}_0 = ((m_i, \rho_i))_{i=1}^N$ を出力する．

編集結果の認証． Sanitize は，外部アプリケーションから与えられた各 $i \in I$ の編集後値 m'_i について $\mathsf{Pol}(j, i, m_i, m'_i) = 1$ を確認する．コミットメントを C_i から $C'_i = \mathsf{Com}(m'_i; \rho'_i)$ へ更新し，Merkle root を r_{in} から r_{out} へ更新する．また，公開ステートメント x_i とウィットネス $w_i = (m_i, \rho_i, \rho'_i)$ から PC-NIZK 証明 π_i を生成する．これらを含む $\widehat{\ell}_k$ に対して

$$\sigma_j \leftarrow \mathsf{Sig.Sign}(sk_j, \mathsf{anc} \| \widehat{\ell}_k \| \mathsf{segid} \| \mathsf{ppid})$$

を生成し，履歴エントリ $\ell_k = (\widehat{\ell}_k, \sigma_j)$ を追加する．すべての対象セグメントを処理した後，更新後コンテンツと最終ルートに対する CR-NIZK 証明 π'_{cr} を生成し，$(\Sigma', \mathsf{open}')$ を出力する．

検証． Verify は，作成者署名と，現在コンテンツ M および現在ルート r に対する CR-NIZK 証明を検証する．さらに，CheckHistory を用いて，初期ルート r_0 から現在ルート r までの各履歴エントリについて，ルートの連続性，Merkle 認証パス，ルート更新，PC-NIZK 証明，および編集者署名を検証する．すべての検証に成功し，履歴の最終ルートが r に一致する場合に 1 を出力する．形式的な各アルゴリズムを付録 A.3 に示す．

6. 安全性

6.1 安全性要件

CAPC-SS には，作成者による初期認証，編集履歴，現在コンテンツとの対応，および編集前情報の秘匿性を保護するため，以下の 5 つの安全性を要求する．形式的な安全性定義は付録 A.4 に示し，以下では直感的な定義を述べる．

定義 5 (作成者アンカー不変性)．作成者が生成していない作成者アンカーおよび関連情報を含む認証情報を，検証者に受理させることができない．

定義 6 (コンテンツ・ルート整合性)．検証者に受理される現在コンテンツ M と現在ルート r について，(M, r) が *4* 章で定義したコンテンツ・ルート整合関係 $\mathcal{R}^{\mathsf{cr}}_{\mathsf{Com}}$ を満たす．

定義 7 (編集者認証性)．秘密鍵が漏えいしていない編集者 j について，当該編集者が生成していない新しい履歴エントリを j の有効な署名付きで検証者に受理させることができない．

定義 8 (ポリシー準拠性)．検証者に受理される各履歴エントリについて，対応する *PC-NIZK* 公開ステートメントがポリシー準拠関係 $\mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}}$ を満たすウィットネスを持つ．

定義 9 (編集前情報の秘匿性)．同一の公開情報および最終コンテンツを与える二つの有効な編集系列について，公開される認証情報と履歴から，どちらの非公開編集前値系列が用いられたかを計算量的に識別できない．

編集前情報の秘匿性は外部の検証者に対する性質である．一度公開された編集後値や，公開ポリシーから推測可能な情報は秘匿対象に含めない．また，編集を実行する編

© 2026 Information Processing Society of Japan

集者は，当該編集に必要な編集前値とコミットメントの開示情報を知るものとする．

6.2 一般構成の安全性

定理 1 (一般構成の安全性)．作成者署名と編集者署名が *EUF-CMA* 安全であり，Com が秘匿性と束縛性を満たし，*Merkle* 木に用いるハッシュ関数 H が衝突困難であり，*PC-NIZK* および *CR-NIZK* が知識抽出可能性とゼロ知識性を満たすと仮定する．このとき，*5.3* 章の一般構成は，作成者アンカー不変性，コンテンツ・ルート整合性，編集者認証性，ポリシー準拠性，および編集前情報の秘匿性を満たす．

証明を付録 A.5 に示す．

7. 効率評価と実現可能性

本章では，CAPC-SS の漸近的な効率を評価するとともに，一般構成に固有の構成要素である PC-NIZK および CR-NIZK を，既存のコミットメント方式と抽出可能 NIZK から実現できることを示す．

7.1 CAPC-SS の漸近効率

履歴エントリ数を t，セグメント数を N，CR-NIZK のサイズを $|\pi_{\mathrm{cr}}|$ とする．作成者アンカーと現在ルート・履歴長からなる情報は $O(1)$ サイズであり，公開履歴を含む認証情報のサイズは

$$O\big(|\pi_{\mathrm{cr}}| + t\big(|\pi| + |\sigma| + |m'| + |\mathsf{Com}| + |H|\log N\big)\big)$$

である．各履歴エントリは，編集前後のコミットメント，公開編集後値，Merkle 認証パス，入出力ルート，PC-NIZK 証明，編集者署名等を含む．

単純な Sanitize では，非公開の開示情報全体から現在ルートを再確認する $O(N)$ の処理に加え，更新セグメントごとに $O(\log N)$ 回のハッシュ計算，PC-NIZK 証明生成，編集者署名生成を要する．さらに，更新後の現在コンテンツ全体に対する CR-NIZK を 1 回生成する．コミットメント列を認証付きデータ構造として管理すれば，ルートの再確認を差分更新により行うこともできる．Verify は作成者署名と CR-NIZK をそれぞれ 1 回検証し，各履歴エントリについて編集者署名，PC-NIZK 証明，および $O(\log N)$ 回のハッシュ計算を行うため，履歴長 t に対して線形である．CR-NIZK の生成・検証コストは採用する NIZK とセグメント数 N に依存する．

7.2 PC-NIZK および CR-NIZK の実現可能性

位数 q の巡回群 $\mathbb{G}$ と生成元 g, h を用い，$h = g^{\alpha}$ を満たす α が未知であるとする．セグメント値を $\mathbb{Z}_q$ へ符号化し，Pedersen コミットメント $\mathsf{Com}(m;\rho) = g^m h^{\rho}$ を用いる [19]．このコミットメントは完全秘匿性を持ち，離散対数仮定の下で束縛性を満たす．

コミットメント方式とハッシュ関数が多項式時間で計算可能であり，ポリシー Pol が多項式時間で判定可能であるならば，$\mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}}$ および $\mathcal{R}^{\mathsf{cr}}_{\mathsf{Com}}$ は NP 関係である．したがって，完全性，知識抽出可能性，およびゼロ知識性を満たす抽出可能 NIZK をそれぞれの関係に適用することで，PC-NIZK および CR-NIZK を実現できる．

これらを EUF-CMA 安全なデジタル署名方式，Pedersen コミットメント，および衝突困難なハッシュ関数とともに 5 章の一般構成へ適用することで，CAPC-SS が得られる．その安全性は定理 1 から従う．両 NIZK の具体的な判定処理は付録 A.2 に示す．

8. 来歴情報管理への応用

8.1 C2PA への統合

C2PA では，検証者は各段階のマニフェストについてクレーム署名を検証し，バインディングによってマニフェストとアセットとの対応を確認することで，作成・編集・配信に関する署名付き来歴を検証する．CAPC-SS はこれを補完し，各編集のポリシー準拠性と，作成者アンカーから現在コンテンツまでの対応を検証する．

両者の役割は次のように分離される．C2PA のバインディングは現在のアセットとマニフェストとの対応を検証する．一方，CR-NIZK は，アセットから導出した現在コンテンツ $M = (m_1, \ldots, m_N)$ と CAPC-SS の現在ルート r との対応を検証する．さらに，CAPC-SS の履歴検証により，作成者アンカー中の初期ルート r_0 から r までの各編集について，Merkle root の更新，PC-NIZK 証明，および編集者署名を検証する．これにより，現在アセット，現在ルート，作成者アンカーを一貫して検証できる．

C2PA へ統合する場合，CAPC-SS 用アサーションを追加する．初期マニフェストでは，作成者アンカー anc，作成者署名 σ_C，ポリシー Pol，編集者公開鍵集合 $\mathcal{PK}$，セグメント化・正規化規則識別子 segid，および公開パラメータ識別子 ppid を保持する．各編集後のマニフェストでは，現在ルート r_k，CR-NIZK 証明 $\pi_{\mathrm{cr},k}$，および対応する履歴エントリを保持するか，それらへのハッシュ付き参照を保持する．編集前値 m_i やコミットメント乱数 ρ_i は格納しない．

CAPC-SS 用アサーションを C2PA クレームから参照させ，クレーム署名によって保護することで，C2PA の来歴情報と CAPC-SS の検証情報を同一のマニフェスト上で扱うことができる．また，segid を作成者署名へ結び付けることで，後続の処理段階でセグメントの解釈が変更されることを防ぐ．

8.2 検証

CAPC-SS を組み込んだ C2PA 来歴は，次の三段階で検証する．

© 2026 Information Processing Society of Japan

(1) C2PA の通常検証により，現在アセットとマニフェストとのバインディング，クレーム署名，および来歴の整合性を確認する．

(2) segid で識別される規則に従って現在アセットから M を導出し，$x_{\mathrm{cr}} = (M, r)$ に対する CR-NIZK 証明 π_{cr} を検証する．

(3) CAPC-SS の作成者署名と編集履歴を検証し，r_0 から r までの各 Merkle root 更新，PC-NIZK 証明，および編集者署名を確認する．

ここで，C2PA のクレーム署名が正しく検証できることと，編集が作成者のポリシーに準拠していることは異なる．例えば，正規の C2PA 署名者がポリシーで許可されていない編集を行ってマニフェストを正しく署名した場合でも，C2PA の検証は成立し得る一方，対応する PC-NIZK の検証は成立しない．したがって CAPC-SS は，編集そのものを禁止するのではなく，受理された現在コンテンツが，作成者の認証したコンテンツを起点としてポリシーに準拠した編集を経て生成されたことを検証可能にする．

参考文献

[1] Coalition for Content Provenance and Authenticity. C2PA technical specification, version 2.4. Official Specification, 2026. April 2026.

[2] Coalition for Content Provenance and Authenticity. C2PA and content credentials explainer, version 2.4. Official Explainer, 2026. April 2026.

[3] Adobe. Content credentials overview. Adobe Creative Cloud Help, 2026. Last updated April 7, 2026. Accessed August 7, 2026.

[4] OpenAI. C2PA and SynthID in OpenAI-generated images. OpenAI Help Center, 2026. Accessed August 7, 2026.

[5] Google. How pixel and android are bringing a new level of trust to your images with C2PA content credentials. Google Security Blog, September 2025. September 10, 2025. Accessed August 7, 2026.

[6] Giuseppe Ateniese, Daniel H. Chou, Breno de Medeiros, and Gene Tsudik. Sanitizable signatures. In *Computer Security – ESORICS 2005*, volume 3679 of *Lecture Notes in Computer Science*, pages 159–177. Springer, 2005.

[7] Christina Brzuska, Marc Fischlin, Tobias Freudenreich, Anja Lehmann, Marcus Page, Jakob Schelbert, Dominique Schröder, and Florian Volk. Security of sanitizable signatures revisited. In *Public Key Cryptography – PKC 2009*, volume 5443 of *Lecture Notes in Computer Science*, pages 317–336. Springer, 2009.

[8] Christina Brzuska, Marc Fischlin, Anja Lehmann, and Dominique Schröder. Unlinkability of sanitizable signatures. In *Public Key Cryptography – PKC 2010*, volume 6056 of *Lecture Notes in Computer Science*, pages 444–461. Springer, 2010.

[9] Sébastien Canard and Amandine Jambert. On extended sanitizable signature schemes. In *Topics in Cryptology – CT-RSA 2010*, volume 5985 of *Lecture Notes in Computer Science*, pages 179–194. Springer, 2010.

[10] Sébastien Canard, Amandine Jambert, and Roch Lescuyer. Sanitizable signatures with several signers and sanitizers. In *Progress in Cryptology – AFRICACRYPT 2012*, volume 7374 of *Lecture Notes in Computer Science*, pages 35–52. Springer, 2012.

[11] Xavier Bultel and Pascal Lafourcade. Unlinkable and strongly accountable sanitizable signatures from verifiable ring signatures. In *Cryptology and Network Security – CANS 2017*, Lecture Notes in Computer Science, pages 203–226. Springer, 2017.

[12] Kai Samelin and Daniel Slamanig. Policy-based sanitizable signatures. In *Topics in Cryptology – CT-RSA 2020*, volume 12006 of *Lecture Notes in Computer Science*, pages 538–563. Springer, 2020.

[13] Ismail Afia and Riham AlTawy. Unlinkable policy-based sanitizable signatures. In *Topics in Cryptology – CT-RSA 2023*, Lecture Notes in Computer Science, pages 191–221. Springer, 2023.

[14] Angèle Bossuat and Xavier Bultel. Unlinkable and invisible γ-sanitizable signatures. In *Applied Cryptography and Network Security – ACNS 2021*, Lecture Notes in Computer Science, pages 251–283. Springer, 2021.

[15] Karthik Nandakumar, Nalini K. Ratha, and Sharathchandra Pankanti. Proving multimedia integrity using sanitizable signatures recorded on blockchain. In *Proceedings of the ACM Workshop on Information Hiding and Multimedia Security – IH&MMSec 2019*, pages 151–160. ACM, 2019.

[16] Assa Naveh and Eran Tromer. Photoproof: Cryptographic image authentication for any set of permissible transformations. In *IEEE Symposium on Security and Privacy – S&P 2016*, pages 255–271. IEEE, 2016.

[17] Alessandro Chiesa and Eran Tromer. Proof-carrying data and hearsay arguments from signature cards. In *Innovations in Computer Science (ICS 2010)*, pages 310–331. Tsinghua University Press, 2010.

[18] Trisha Datta, Binyi Chen, and Dan Boneh. Veritas: Verifying image transformations at scale. In *IEEE Symposium on Security and Privacy – S&P 2025*, pages 4606–4623. IEEE, 2025.

[19] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Advances in Cryptology – CRYPTO 1991*, volume 576 of *Lecture Notes in Computer Science*, pages 129–140. Springer, 1992.

[20] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology – CRYPTO '87*, volume 293 of *Lecture Notes in Computer Science*, pages 369–378. Springer, 1988.

[21] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 305–326. Springer, 2016.

[22] Jens Groth and Mary Maller. Snarky signatures: Minimal signatures of knowledge from simulation-extractable SNARKs. In *Advances in Cryptology – CRYPTO 2017, Part II*, volume 10402 of *Lecture Notes in Computer Science*, pages 581–612. Springer, 2017.

付　　録

A.1 作成者アンカーの位置付け

本稿の作成者アンカーと C2PA における公開鍵証明書

© 2026 Information Processing Society of Japan

のトラストアンカーは，その位置付けが異なる．C2PA のトラストアンカーは，クレーム署名者の証明書を検証し，署名者に対する信頼を確立するための基点である．これに対して作成者アンカーは，作成者が最初に認証したコンテンツと編集ポリシーを，後続の編集を検証するための固定的な基準として保持する．両者は異なる役割を持つため，CAPC-SS は C2PA のトラストモデルと併用でき，C2PA による署名者の信頼性検証に対して，作成者を起点とした編集のポリシー準拠性検証を追加できる．

A.2 PC-NIZK および CR-NIZK の判定処理

本章では，Section 7.2 で述べた実現方法について，PC-NIZK および CR-NIZK が証明する判定処理を示す．

基礎となる NIZK. 多項式サイズの算術回路の充足可能性に対して，完全性，知識抽出可能性，およびゼロ知識性を満たす抽出可能 NIZK を Π_{zk} とする．このような証明系の代表例として，Groth による算術回路充足可能性に対する pre-processing zk-SNARK がある [21]．また，Groth と Maller は，ゼロ知識性に加えて simulation-extractability を満たす SE-SNARK を構成している [22]．後者の simulation-extractability は，本稿で要求する知識抽出可能性より強い性質である．したがって，これらの方式を，各方式が前提とする安全性モデルおよび仮定の下で Π_{zk} の候補として利用できる．

PC-NIZK ではポリシー準拠関係 $\mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}}$ の判定処理を表す回路 $\mathcal{C}_{\mathsf{pc}}$ を用い，CR-NIZK ではコンテンツ・ルート整合関係 $\mathcal{R}^{\mathsf{cr}}_{\mathsf{Com}}$ の判定処理を表す回路 $\mathcal{C}_{\mathsf{cr}}$ を用いる．回路の記述，証明鍵，検証鍵，共通参照文字列等の採用する NIZK に固有の情報は，それぞれ pp_{pc} および pp_{cr} に含まれるものとし，本文の構文では記法から省略している．

PC-NIZK の判定処理. PC-NIZK の判定回路 $\mathcal{C}_{\mathsf{pc}}$ は，本文で定義した公開ステートメント x_i とウィットネス w_i を入力として受け取る．これらはそれぞれ

$$x_i = (\mathsf{sid}, j, i, C_i, C'_i, m'_i, \mathsf{Pol}, r_{\mathrm{in}}, r_{\mathrm{out}}, \mathsf{ppid}),$$
$$w_i = (m_i, \rho_i, \rho'_i)$$

である．7.2 章の Pedersen コミットメント $\mathsf{Com}(m;\rho) = g^m h^\rho$ を用いる場合，$\mathcal{C}_{\mathsf{pc}}$ は

$$C_i = g^{m_i} h^{\rho_i},$$
$$C'_i = g^{m'_i} h^{\rho'_i},$$
$$\mathsf{Pol}(j, i, m_i, m'_i) = 1$$

がすべて成立する場合に限り 1 を出力する．これにより，証明者は編集前値 m_i とコミットメントの開示情報 ρ_i, ρ'_i を公開することなく，C_i と C'_i がそれぞれ編集前後値に対する正しいコミットメントであり，編集がポリシーを満たすことを証明する．

sid，r_{in}，r_{out}，および ppid は，証明対象となる履歴の文脈を識別するために公開ステートメントへ含める．Merkle 認証パスとルート更新の正当性は $\mathcal{C}_{\mathsf{pc}}$ では判定せず，CheckHistory が通常のハッシュ検証によって確認する．また，公開ステートメント，PC-NIZK 証明，Merkle 情報等を含む履歴エントリ全体は編集者署名によって認証される．

CR-NIZK の判定処理. CR-NIZK の判定回路 $\mathcal{C}_{\mathsf{cr}}$ は，公開ステートメント $x_{\mathrm{cr}} = (M, r)$ とウィットネス $w_{\mathrm{cr}} = (\rho_1, \ldots, \rho_N)$ を入力として受け取る．ここで，$M = (m_1, \ldots, m_N)$ は検証対象となる現在コンテンツのセグメント列であり，r は CAPC-SS の認証情報に含まれる現在ルートである．

$\mathcal{C}_{\mathsf{cr}}$ は，各 $i \in \{1, \ldots, N\}$ について $C_i = g^{m_i} h^{\rho_i}$ を計算し，同一のハッシュ関数，葉の順序，および木の構成規則を用いて Merkle root を再計算する．その結果が

$$r = \mathsf{MerkleRoot}_H(C_1, \ldots, C_N)$$

を満たす場合に限り 1 を出力する．これにより，検証者はコミットメント乱数 $\rho_1, \ldots, \rho_N$ を知ることなく，公開された現在コンテンツ M が認証情報中の現在ルート r に対応することを確認できる．葉の順序，セグメントの符号化，およびセグメント数が完全二分木を構成しない場合の処理は，segid と公開パラメータによって一意に定める．

ポリシーの回路表現. 利用可能なポリシーを設定時に固定する場合には，Pol の判定処理を $\mathcal{C}_{\mathsf{pc}}$ へ直接組み込むことができる．コンテンツごとに異なるポリシーを扱う場合には，ポリシーの回路記述を公開入力として評価する汎用回路を用いるか，ポリシーごとに生成した証明パラメータを ppid によって識別する方法を利用できる．いずれの場合も，ポリシー，ppid，および segid は作成者署名の対象に含まれるため，後続の編集者がこれらを置換することはできない．

A.3 CAPC-SS の詳細アルゴリズム

本節では，5.3 節の一般構成における Sign，Sanitize，Verify，および補助関数 CheckHistory を示す．以下では $pp = (pp_{\mathrm{pc}}, pp_{\mathrm{cr}}, pp_{\mathrm{com}}, H, N, \mathsf{ppid})$ とし，各構成要素の公開パラメータを暗黙に用いる．また，$\|$ は各要素を一意に復元できる正規な符号化を表す．履歴エントリ数を t とし，編集履歴を $\mathsf{hist} = (\ell_1, \ldots, \ell_t)$ とする．初期状態では $t = 0$ かつ $\mathsf{hist} = \emptyset$ である．

Setup は PC-NIZK，CR-NIZK，およびコミットメント方式の公開パラメータを生成し，ppid とともに pp へ格納する．CKeyGen と OKeyGen は Sig.KeyGen を実行し，それぞれ作成者と編集者の署名鍵対を生成する．これらは構成要素のアルゴリズムを直接実行するため，以下では擬似コードを省略する．Sign を Algorithm 1，Sanitize を

© 2026 Information Processing Society of Japan

Algorithm 2，Verify を Algorithm 3 に示す．また，Verify の内部関数 CheckHistory を Algorithm 4 に示す．

Algorithm 1: Sign$(pp, sk_C, M_0, \mathsf{Pol}, \mathcal{PK}, \mathsf{segid})$

$M_0 = (m_1, \ldots, m_N)$，Pol，$\mathcal{PK}$，segid が整形式でなく，または $\mathcal{PK}$ 中の編集者識別子が一意でなければ $\perp$ を返す;
for $i = 1$ **to** N **do**
  乱数 ρ_i を選び，$C_i \leftarrow \mathsf{Com}(m_i; \rho_i)$ とする;
$r_0 \leftarrow \mathsf{MerkleRoot}_H(C_1, \ldots, C_N)$;
sid を生成し，$\mathsf{anc} \leftarrow (r_0, N, \mathsf{sid})$ とする;
$\sigma_C \leftarrow \mathsf{Sig.Sign}(sk_C, \mathsf{anc} \| \mathsf{Pol} \| \mathcal{PK} \| \mathsf{segid} \| \mathsf{ppid})$;
$x_{\mathrm{cr},0} \leftarrow (M_0, r_0)$;
$\pi_{\mathrm{cr},0} \leftarrow \mathsf{CRNIZK.Prove}(pp_{\mathrm{cr}}, x_{\mathrm{cr},0}, (\rho_1, \ldots, \rho_N))$;
$\Sigma_0 \leftarrow (\mathsf{anc}, \sigma_C, \mathsf{Pol}, \mathcal{PK}, \mathsf{segid}, (r_0, 0), \pi_{\mathrm{cr},0}, \emptyset)$;
$\mathsf{open}_0 \leftarrow ((m_i, \rho_i))_{i=1}^N$;
return $(\Sigma_0, \mathsf{open}_0)$;

Algorithm 2: Sanitize$(pp, sk_j, j, \Sigma, \mathsf{open}, I, \Delta)$

$\Sigma = (\mathsf{anc}, \sigma_C, \mathsf{Pol}, \mathcal{PK}, \mathsf{segid}, (r, t), \pi_{\mathrm{cr}}, \mathsf{hist})$，$\mathsf{anc} = (r_0, N_{\mathrm{anc}}, \mathsf{sid})$ と解析する;
解析に失敗し，または $N_{\mathrm{anc}} \neq N$，$t \neq |\mathsf{hist}|$，$\emptyset = I \not\subseteq \{1, \ldots, N\}$，$\Delta$ が I 上で定義されていない，もしくは $(j, pk_j) \in \mathcal{PK}$ が一意でなければ $\perp$ を返す;
for $h = 1$ **to** N **do**
  $\mathsf{open}[h] = (m_h, \rho_h)$ と解析し，$C_h \leftarrow \mathsf{Com}(m_h; \rho_h)$ とする;
解析に失敗し，または $\mathsf{MerkleRoot}_H(C_1, \ldots, C_N) \neq r$ ならば $\perp$ を返す;
$\mathsf{hist}' \leftarrow \mathsf{hist}$，$\mathsf{open}' \leftarrow \mathsf{open}$，$r_{\mathrm{cur}} \leftarrow r$，$t_{\mathrm{cur}} \leftarrow t$;
foreach $i \in I$ **do**
  $\mathsf{open}'[i] = (m_i, \rho_i)$，$m'_i \leftarrow \Delta(i)$ とする;
  $\mathsf{Pol}(j, i, m_i, m'_i) \neq 1$ ならば $\perp$ を返す;
  $C_i \leftarrow \mathsf{Com}(m_i; \rho_i)$;
  $\mu_i \leftarrow \mathsf{MerklePath}_H(i, C_1, \ldots, C_N)$;
  乱数 ρ'_i を選び，$C'_i \leftarrow \mathsf{Com}(m'_i; \rho'_i)$ とする;
  $r_{\mathrm{out}} \leftarrow \mathsf{UpdateMerkle}_H(r_{\mathrm{cur}}, i, C_i, C'_i, \mu_i)$;
  $x_i \leftarrow (\mathsf{sid}, j, i, C_i, C'_i, m'_i, \mathsf{Pol}, r_{\mathrm{cur}}, r_{\mathrm{out}}, \mathsf{ppid})$;
  $\pi_i \leftarrow \mathsf{PCNIZK.Prove}(pp_{\mathrm{pc}}, x_i, (m_i, \rho_i, \rho'_i))$;
  $k \leftarrow t_{\mathrm{cur}} + 1$;
  $\widehat{\ell}_k \leftarrow (j, i, C_i, C'_i, m'_i, \mu_i, r_{\mathrm{cur}}, r_{\mathrm{out}}, \pi_i)$;
  $\sigma_{j,k} \leftarrow \mathsf{Sig.Sign}(sk_j, \mathsf{anc} \| \widehat{\ell}_k \| \mathsf{segid} \| \mathsf{ppid})$;
  $\ell_k \leftarrow (\widehat{\ell}_k, \sigma_{j,k})$ を hist' の末尾に追加する;
  $\mathsf{open}'[i] \leftarrow (m'_i, \rho'_i)$，$C_i \leftarrow C'_i$，$r_{\mathrm{cur}} \leftarrow r_{\mathrm{out}}$，$t_{\mathrm{cur}} \leftarrow k$ と更新する;
open' の値成分を順に並べた列を M'，乱数成分を順に並べた列を $\boldsymbol{\rho}'$ とする;
$x'_{\mathrm{cr}} \leftarrow (M', r_{\mathrm{cur}})$;
$\pi'_{\mathrm{cr}} \leftarrow \mathsf{CRNIZK.Prove}(pp_{\mathrm{cr}}, x'_{\mathrm{cr}}, \boldsymbol{\rho}')$;
$\Sigma' \leftarrow (\mathsf{anc}, \sigma_C, \mathsf{Pol}, \mathcal{PK}, \mathsf{segid}, (r_{\mathrm{cur}}, t_{\mathrm{cur}}), \pi'_{\mathrm{cr}}, \mathsf{hist}')$;
return $(\Sigma', \mathsf{open}')$;

Algorithm 3: Verify(pp, pk_C, M, Σ)

$\Sigma = (\mathsf{anc}, \sigma_C, \mathsf{Pol}, \mathcal{PK}, \mathsf{segid}, (r, t), \pi_{\mathrm{cr}}, \mathsf{hist})$，$\mathsf{anc} = (r_0, N_{\mathrm{anc}}, \mathsf{sid})$ と解析する;
解析に失敗し，または $N_{\mathrm{anc}} \neq N$，$|M| \neq N$，$t \neq |\mathsf{hist}|$，もしくは $\mathcal{PK}$ 中の編集者識別子が一意でなければ 0 を返す;
$\mathsf{Sig.Verify}(pk_C, \mathsf{anc} \| \mathsf{Pol} \| \mathcal{PK} \| \mathsf{segid} \| \mathsf{ppid}, \sigma_C) \neq 1$ ならば 0 を返す;
$x_{\mathrm{cr}} \leftarrow (M, r)$;
$\mathsf{CRNIZK.Verify}(pp_{\mathrm{cr}}, x_{\mathrm{cr}}, \pi_{\mathrm{cr}}) \neq 1$ ならば 0 を返す;
return $\mathsf{CheckHistory}(pp, \mathsf{hist}, r_0, r, \mathsf{sid}, \mathsf{Pol}, \mathcal{PK}, \mathsf{segid}, \mathsf{anc})$;

Algorithm 4: CheckHistory$(pp, \mathsf{hist}, r_0, r, \mathsf{sid}, \mathsf{Pol}, \mathcal{PK}, \mathsf{segid}, \mathsf{anc})$

$r_{\mathrm{cur}} \leftarrow r_0$;
foreach $\ell_k \in \mathsf{hist}$ **do**
  $\ell_k = (\widehat{\ell}_k, \sigma_{j,k})$，$\widehat{\ell}_k = (j, i, C_i, C'_i, m'_i, \mu_i, r_{\mathrm{in}}, r_{\mathrm{out}}, \pi_i)$ と解析する;
  解析に失敗し，または $i \notin \{1, \ldots, N\}$ ならば 0 を返す;
  $r_{\mathrm{in}} \neq r_{\mathrm{cur}}$ ならば 0 を返す;
  $(j, pk_j) \in \mathcal{PK}$ が一意でなければ 0 を返す;
  $\mathsf{VerifyPath}_H(r_{\mathrm{in}}, i, C_i, \mu_i) \neq 1$ ならば 0 を返す;
  $\mathsf{UpdateMerkle}_H(r_{\mathrm{in}}, i, C_i, C'_i, \mu_i) \neq r_{\mathrm{out}}$ ならば 0 を返す;
  $x_i \leftarrow (\mathsf{sid}, j, i, C_i, C'_i, m'_i, \mathsf{Pol}, r_{\mathrm{in}}, r_{\mathrm{out}}, \mathsf{ppid})$;
  $\mathsf{PCNIZK.Verify}(pp_{\mathrm{pc}}, x_i, \pi_i) \neq 1$ ならば 0 を返す;
  $\mathsf{Sig.Verify}(pk_j, \mathsf{anc} \| \widehat{\ell}_k \| \mathsf{segid} \| \mathsf{ppid}, \sigma_{j,k}) \neq 1$ ならば 0 を返す;
  $r_{\mathrm{cur}} \leftarrow r_{\mathrm{out}}$;
return $r_{\mathrm{cur}} = r$ **のとき** *1*，**それ以外は** *0*;

A.4 CAPC-SS の形式的な安全性定義

本章では，6 章で述べた CAPC-SS の 5 つの安全性を形式的に定義する．

A.4.1 共通設定とオラクル

認証情報 $\Sigma = (\mathsf{anc}, \sigma_C, \mathsf{Pol}, \mathcal{PK}, \mathsf{segid}, (r, t), \pi_{\mathrm{cr}}, \mathsf{hist})$ について，作成者アンカーを $\mathsf{anc} = (r_0, N, \mathsf{sid})$ とする．作成者署名の対象を $\mathsf{msg}_C(\Sigma) = \mathsf{anc} \| \mathsf{Pol} \| \mathcal{PK} \| \mathsf{segid} \| \mathsf{ppid}$ とする．また，履歴エントリ $\ell = (\widehat{\ell}, \sigma_j)$ に対する編集者署名の対象を $\mathsf{msg}_E(\Sigma, \widehat{\ell}) = \mathsf{anc} \| \widehat{\ell} \| \mathsf{segid} \| \mathsf{ppid}$ とする．

プライバシー試行を除き，攻撃者 $\mathcal{A}$ は最初にセグメント数 N と編集者識別子集合 $\mathcal{J}$ を出力する．チャレンジャは $pp \leftarrow \mathsf{Setup}(1^\lambda, N)$，$(sk_C, pk_C) \leftarrow \mathsf{CKeyGen}(pp)$ を実行し，各 $j \in \mathcal{J}$ について $(sk_j, pk_j) \leftarrow \mathsf{OKeyGen}(pp, j)$ を生成する．編集者公開鍵集合を $\mathcal{PK} = \{(j, pk_j) : j \in \mathcal{J}\}$ とし，$pp, pk_C, \mathcal{PK}$ を $\mathcal{A}$ へ与える．また，$Q_C \leftarrow \emptyset$，$Q_E \leftarrow \emptyset$，$\mathcal{J}_{\mathsf{corr}} \leftarrow \emptyset$ と初期化する．

攻撃者は，対象とする安全性に応じて以下のオラクルへ適応的に問い合わせることができる．

作成者署名オラクル $\mathcal{O}_{\mathsf{Sign}}$．入力 $(M_0, \mathsf{Pol}, \mathsf{segid})$ に対して

$$(\Sigma_0, \mathsf{open}_0) \leftarrow \mathsf{Sign}(pp, sk_C, M_0, \mathsf{Pol}, \mathcal{PK}, \mathsf{segid})$$

を実行する．Σ_0 に含まれる作成者署名対象 $\mathsf{msg}_C(\Sigma_0)$ を

© 2026 Information Processing Society of Japan

Q_C へ記録し，$(\Sigma_0, \mathsf{open}_0)$ を返す．

墨塗りオラクル $\mathcal{O}_{\mathsf{San}}$**．** 入力 $(j, \Sigma, \mathsf{open}, I, \Delta)$ に対し，$j \in \mathcal{J} \setminus \mathcal{J}_{\mathsf{corr}}$ であれば

$$(\Sigma', \mathsf{open}') \leftarrow \mathsf{Sanitize}(pp, sk_j, j, \Sigma, \mathsf{open}, I, \Delta)$$

を実行する．Sanitize が $\perp$ を出力した場合は $\perp$ を返す．それ以外の場合，新たに生成された各履歴エントリ $\ell = (\widehat{\ell}, \sigma_j)$ について $(j, \mathsf{msg}_E(\Sigma, \widehat{\ell}))$ を Q_E へ記録し，$(\Sigma', \mathsf{open}')$ を返す．

鍵漏えいオラクル $\mathcal{O}_{\mathsf{Corrupt}}$**．** 入力 $j \in \mathcal{J}$ に対して sk_j を返し，$\mathcal{J}_{\mathsf{corr}} \leftarrow \mathcal{J}_{\mathsf{corr}} \cup \{j\}$ とする．

A.4.2 形式的な安全性定義

定義 5 (作成者アンカー不変性)**.** 攻撃者 $\mathcal{A}$ は上記のオラクルへ適応的に問い合わせた後，現在コンテンツ M^* と認証情報

$$\Sigma^* = (\mathsf{anc}^*, \sigma_C^*, \mathsf{Pol}^*, \mathcal{PK}^*, \mathsf{segid}^*, (r^*, t^*), \pi_{\mathrm{cr}}^*, \mathsf{hist}^*)$$

を出力する．$\mathsf{Verify}(pp, pk_C, M^*, \Sigma^*) = 1$ かつ $\mathsf{msg}_C(\Sigma^*) \notin Q_C$ が成立するとき，$\mathcal{A}$ の勝利とする．任意の PPT 攻撃者 $\mathcal{A}$ の勝利確率が λ について無視できるとき，*CAPC-SS* は作成者アンカー不変性を満たすという．

定義 6 (コンテンツ・ルート整合性)**.** 攻撃者 $\mathcal{A}$ が現在コンテンツ M^* と認証情報 Σ^* を出力する．Σ^* 中の現在ルートを r^* とし，$x_{\mathrm{cr}}^* = (M^*, r^*)$ とする．$\mathsf{Verify}(pp, pk_C, M^*, \Sigma^*) = 1$ かつ $(x_{\mathrm{cr}}^*, w_{\mathrm{cr}}^*) \in \mathcal{R}_{\mathsf{Com}}^{\mathsf{cr}}$ を満たすウィットネス w_{cr}^* が存在しないとき，$\mathcal{A}$ の勝利とする．任意の PPT 攻撃者 $\mathcal{A}$ の勝利確率が λ について無視できるとき，*CAPC-SS* はコンテンツ・ルート整合性を満たすという．

定義 7 (編集者認証性)**.** 攻撃者 $\mathcal{A}$ は $\mathcal{O}_{\mathsf{Sign}}$，$\mathcal{O}_{\mathsf{San}}$，$\mathcal{O}_{\mathsf{Corrupt}}$ へ適応的に問い合わせた後，(M^*, Σ^*) を出力する．

$\mathsf{Verify}(pp, pk_C, M^*, \Sigma^*) = 1$ かつ hist^* 中に $\ell^* = (\widehat{\ell}^*, \sigma_{j^*}^*)$ が存在して，$j^* \notin \mathcal{J}_{\mathsf{corr}}$ および $\big(j^*, \mathsf{msg}_E(\Sigma^*, \widehat{\ell}^*)\big) \notin Q_E$ が成立するとき，$\mathcal{A}$ の勝利とする．任意の PPT 攻撃者 $\mathcal{A}$ の勝利確率が λ について無視できるとき，*CAPC-SS* は編集者認証性を満たすという．

定義 8 (ポリシー準拠性)**.** 攻撃者 $\mathcal{A}$ は上記のオラクルへ適応的に問い合わせた後，(M^*, Σ^*) を出力する．以下では $\mathsf{Verify}(pp, pk_C, M^*, \Sigma^*) = 1$ かつ $\mathsf{msg}_C(\Sigma^*) \in Q_C$ である場合のみを考える．

$\mathsf{anc}^* = (r_0^*, N^*, \mathsf{sid}^*)$ とし，hist^* 中の履歴エントリ $\ell_k^* = (\widehat{\ell}_k^*, \sigma_{j_k}^*)$ を $\widehat{\ell}_k^* = (j_k^*, i_k^*, C_k^*, C_k'^*, m_k'^*, \mu_k^*, r_{\mathrm{in},k}^*, r_{\mathrm{out},k}^*, \pi_k^*)$ と解析する．対応する *PC-NIZK* 公開ステートメントを

$$x_k^* = (\mathsf{sid}^*, j_k^*, i_k^*, C_k^*, C_k'^*, m_k'^*, \mathsf{Pol}^*, r_{\mathrm{in},k}^*, r_{\mathrm{out},k}^*, \mathsf{ppid})$$

とする．ある k について，$(x_k^*, w_k^*) \in \mathcal{R}_{\mathsf{Com}}^{\mathsf{pc}}$ を満たすウィットネス w_k^* が存在しないとき，$\mathcal{A}$ の勝利とする．任意の PPT 攻撃者 $\mathcal{A}$ の勝利確率が λ について無視できるとき，*CAPC-SS* はポリシー準拠性を満たすという．

なお，Verify が受理する場合，各履歴エントリに含まれる PC-NIZK 証明は既に検証を通過しているため，上記の勝利条件では PCNIZK.Verify の条件を重複して記載していない．

編集前情報の秘匿性については，他の安全性とは異なり，二つの編集系列を比較する識別試行によって定義する．

定義 9 (編集前情報の秘匿性)**.** $b \in \{0,1\}$ に対する試行 $\mathsf{Exp}_{\text{CAPC-SS},\mathcal{A}}^{\mathsf{priv}\text{-}b}(1^\lambda)$ を次のように定義する．

(1) 攻撃者 $\mathcal{A}$ は $(N, M_0^{(0)}, M_0^{(1)}, \mathsf{Pol}, \mathsf{segid}, \mathcal{J}, \mathcal{E})$ を出力する．ここで $\mathcal{E} = ((j_1, I_1, \Delta_1), \ldots, (j_T, I_T, \Delta_T))$ を編集スケジュールとする．

(2) $|M_0^{(0)}| = |M_0^{(1)}| = N$ とする．$\mathcal{E}$ は両側について同一の編集者列，編集対象集合列，および公開編集後値列を指定するものとし，両側のすべての編集が Pol を満たすことを要求する．さらに，$\mathcal{E}$ を適用した後の最終コンテンツが $M_T^{(0)} = M_T^{(1)} =: M_T$ となることを要求する．これらの条件を満たさない場合，試行を中止する．

(3) チャレンジャは $pp \leftarrow \mathsf{Setup}(1^\lambda, N)$，$(sk_C, pk_C) \leftarrow \mathsf{CKeyGen}(pp)$ を実行し，各 $j \in \mathcal{J}$ について $(sk_j, pk_j) \leftarrow \mathsf{OKeyGen}(pp, j)$ を生成する．また，$\mathcal{PK} = \{(j, pk_j) : j \in \mathcal{J}\}$ とする．

(4) チャレンジャは $(\Sigma_0^{(b)}, \mathsf{open}_0^{(b)}) \leftarrow \mathsf{Sign}(pp, sk_C, M_0^{(b)}, \mathsf{Pol}, \mathcal{PK}, \mathsf{segid})$ を実行する．

(5) $k = 1, \ldots, T$ について，$\mathcal{E}$ に従って $(\Sigma_k^{(b)}, \mathsf{open}_k^{(b)}) \leftarrow \mathsf{Sanitize}(pp, sk_{j_k}, j_k, \Sigma_{k-1}^{(b)}, \mathsf{open}_{k-1}^{(b)}, I_k, \Delta_k)$ を順に実行する．

(6) 最終的な $(M_T, \Sigma_T^{(b)})$ を $\mathcal{A}$ へ与え，$\mathcal{A}$ は $b' \in \{0,1\}$ を出力する．試行の出力を b' とする．

攻撃者 $\mathcal{A}$ の識別優位性を

$$\mathsf{Adv}_{\text{CAPC-SS},\mathcal{A}}^{\mathsf{priv}}(\lambda) = \Big| \Pr[\mathsf{Exp}_{\text{CAPC-SS},\mathcal{A}}^{\mathsf{priv}\text{-}0}(1^\lambda) = 1] - \Pr[\mathsf{Exp}_{\text{CAPC-SS},\mathcal{A}}^{\mathsf{priv}\text{-}1}(1^\lambda) = 1] \Big|$$

とする．任意の PPT 攻撃者 $\mathcal{A}$ について $\mathsf{Adv}_{\text{CAPC-SS},\mathcal{A}}^{\mathsf{priv}}(\lambda)$ が無視できるとき，*CAPC-SS* は編集前情報の秘匿性を満たすという．

この定義では，両系列で公開されるポリシー，編集者列，編集対象列，編集後値列，セグメント化規則，および最終コンテンツを同一とする．したがって，識別対象となる差は，最終的に秘匿される編集前値と，それらに依存して生成されるコミットメント，Merkle 木，およびゼロ知識証明に限定される．

© 2026 Information Processing Society of Japan

A.5 一般構成の安全性証明

本章では，まず CAPC-SS の構成が正当性を満たすことを示す．そして定理 1 を構成する 5 つの安全性について，A.4 章で定義した各安全性定義を満たすことを示す．

補題 1 (一般構成の正当性)**.** デジタル署名方式 Sig が正当性を満たし，*PC-NIZK* および *CR-NIZK* が完全性を満たすならば，*5.3* 章の一般構成は *CAPC-SS* の正当性を満たす．

Proof. Sign が生成する初期状態は $\mathcal{R}^{\mathsf{cr}}_{\mathsf{Com}}$ を満たす．また，ポリシーを満たす各編集について，Sanitize が生成する (x_i, w_i) は $\mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}}$ を満たし，Merkle 認証パスと入出力ルートは正しく連続する．更新後の最終状態も $\mathcal{R}^{\mathsf{cr}}_{\mathsf{Com}}$ を満たす．したがって，署名方式の正当性と両 NIZK の完全性により，Verify のすべての検証が失敗する確率は無視できる． □

補題 2. 作成者署名が *EUF-CMA* 安全ならば，*CAPC-SS* は作成者アンカー不変性を満たす．

Proof. 攻撃者 $\mathcal{A}$ が作成者アンカー不変性の試行に非無視確率で勝利すると仮定する．定義 5 より，$\mathcal{A}$ が出力した (M^*, Σ^*) について $\mathsf{Verify}(pp, pk_C, M^*, \Sigma^*) = 1$ かつ $\mathsf{msg}_C(\Sigma^*) \notin Q_C$ が成立する．

一方，Verify が受理するためには，Σ^* に含まれる作成者署名 σ^*_C について $\mathsf{Sig.Verify}\big(pk_C, \mathsf{msg}_C(\Sigma^*), \sigma^*_C\big) = 1$ でなければならない．しかし，$\mathsf{msg}_C(\Sigma^*) \notin Q_C$ であるため，このメッセージは作成者署名オラクルを通じて一度も署名されていない．したがって，$(\mathsf{msg}_C(\Sigma^*), \sigma^*_C)$ は作成者署名方式に対する EUF-CMA 偽造を与える．これは作成者署名の EUF-CMA 安全性に矛盾する． □

補題 3. *CR-NIZK* が知識抽出可能性を満たすならば，*CAPC-SS* はコンテンツ・ルート整合性を満たす．

Proof. 攻撃者 $\mathcal{A}$ がコンテンツ・ルート整合性の試行に非無視確率で勝利すると仮定する．勝利条件より，$\mathcal{A}$ が出力した (M^*, Σ^*) について $\mathsf{Verify}(pp, pk_C, M^*, \Sigma^*) = 1$ であるにもかかわらず，Σ^* 中の現在ルート r^* に対する $x^*_{\mathrm{cr}} = (M^*, r^*)$ について，$(x^*_{\mathrm{cr}}, w^*_{\mathrm{cr}}) \in \mathcal{R}^{\mathsf{cr}}_{\mathsf{Com}}$ を満たすウィットネス w^*_{cr} が存在しない．

Verify が受理するためには，Σ^* に含まれる π^*_{cr} について $\mathsf{CRNIZK.Verify}(pp_{\mathrm{cr}}, x^*_{\mathrm{cr}}, \pi^*_{\mathrm{cr}}) = 1$ でなければならない．CR-NIZK の知識抽出可能性から，この受理証明より $w^*_{\mathrm{cr}} = (\rho^*_1, \ldots, \rho^*_N)$ を抽出でき，$r^* = \mathsf{MerkleRoot}_H\big(\mathsf{Com}(m^*_1; \rho^*_1), \ldots, \mathsf{Com}(m^*_N; \rho^*_N)\big)$ が成立する．したがって $(x^*_{\mathrm{cr}}, w^*_{\mathrm{cr}}) \in \mathcal{R}^{\mathsf{cr}}_{\mathsf{Com}}$ であり，有効なウィットネスが存在しないという勝利条件に矛盾する． □

補題 4. 作成者署名および編集者署名が *EUF-CMA* 安全ならば，*CAPC-SS* は編集者認証性を満たす．

Proof. 攻撃者 $\mathcal{A}$ が編集者認証性の試行に無視できない確率で勝利すると仮定する．まず，$\mathsf{msg}_C(\Sigma^*) \notin Q_C$ であれば，$\mathsf{Verify}(pp, pk_C, M^*, \Sigma^*) = 1$ と合わせて，補題 2 と同様に作成者署名に対する EUF-CMA 偽造が得られる．したがって，以下では $\mathsf{msg}_C(\Sigma^*) \in Q_C$ の場合を考えれば十分である．

編集者認証性の勝利条件より，$\mathsf{hist}(\Sigma^*)$ 中に $\ell^* = (\widehat{\ell^*}, \sigma^*_{j^*})$ が存在し，$j^* \notin \mathcal{J}_{\mathsf{corr}}$ かつ $\big(j^*, \mathsf{msg}_E(\Sigma^*, \widehat{\ell^*})\big) \notin Q_E$ が成立する．一方，Verify が受理するためには，$\mathcal{PK}^*$ 中の j^* に対応する公開鍵 pk_{j^*} について $\mathsf{Sig.Verify}\big(pk_{j^*}, \mathsf{msg}_E(\Sigma^*, \widehat{\ell^*}), \sigma^*_{j^*}\big) = 1$ でなければならない．

$j^* \notin \mathcal{J}_{\mathsf{corr}}$ より，j^* の秘密鍵は攻撃者へ漏えいしていない．また，$\big(j^*, \mathsf{msg}_E(\Sigma^*, \widehat{\ell^*})\big) \notin Q_E$ であるため，このメッセージに対する署名は墨塗りオラクルから与えられたものではない．したがって $\big(\mathsf{msg}_E(\Sigma^*, \widehat{\ell^*}), \sigma^*_{j^*}\big)$ は，編集者 j^* の署名方式に対する EUF-CMA 偽造を与える．これは編集者署名の EUF-CMA 安全性に矛盾する． □

補題 5. *PC-NIZK* が知識抽出可能性を満たし，コミットメント方式が束縛性を満たし，*Merkle* 木に用いるハッシュ関数 H が衝突困難ならば，*CAPC-SS* はポリシー準拠性を満たす．

Proof. 攻撃者 $\mathcal{A}$ がポリシー準拠性の試行に非無視確率で勝利すると仮定する．勝利条件より，$\mathsf{Verify}(pp, pk_C, M^*, \Sigma^*) = 1$ であり，受理された履歴中のあるエントリに対応する PC-NIZK 公開ステートメント x^* について，$(x^*, w^*) \in \mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}}$ を満たすウィットネスが存在しない．

一方，Verify が受理するためには，対応する証明 π^* について $\mathsf{PCNIZK.Verify}(pp_{\mathrm{pc}}, x^*, \pi^*) = 1$ でなければならない．PC-NIZK の知識抽出可能性より，この受理証明から $w^* = (m^*, \rho^*, \rho'^*)$ を抽出でき，

$$C^* = \mathsf{Com}(m^*; \rho^*),$$
$$C'^* = \mathsf{Com}(m'^*; \rho'^*),$$
$$\mathsf{Pol}^*(j^*, i^*, m^*, m'^*) = 1$$

が成立する．したがって $(x^*, w^*) \in \mathcal{R}^{\mathsf{pc}}_{\mathsf{Com}}$, となり，有効なウィットネスが存在しないという勝利条件に矛盾する．

さらに，コミットメントの束縛性とハッシュ関数 H の衝突困難性により，抽出された編集前後値は，受理された履歴中のコミットメントおよび Merkle root の遷移と整合する． □

補題 6. コミットメント方式が秘匿性を満たし，*PC-NIZK* および *CR-NIZK* がゼロ知識性を満たすならば，*CAPC-SS* は編集前情報の秘匿性を満たす．

Proof. 定義 9 より，二つの試行 $\mathsf{Exp}^{\mathsf{priv}-0}$ と $\mathsf{Exp}^{\mathsf{priv}-1}$ を

© 2026 Information Processing Society of Japan

考える．両試行では，公開されるポリシー，編集者列，編集対象集合列，公開編集後値列，セグメント化規則，および最終コンテンツ $M_T^{(0)} = M_T^{(1)} =: M_T$ は同一である．したがって，両試行の差は，非公開の編集前値と，それらに依存するコミットメントおよびゼロ知識証明に現れる．

以下のハイブリッドを考える．

Hybrid H_0**.** $\mathsf{Exp}^{\mathsf{priv}-0}$ そのものとする．

Hybrid H_1**.** H_0 における各 PC-NIZK 証明および CR-NIZK 証明を，対応する公開ステートメントに対するシミュレートされた証明へ順に置換する．必要に応じて両 NIZK の公開パラメータもシミュレータが生成する分布へ置換する．PC-NIZK および CR-NIZK のゼロ知識性より，$H_0 \approx_c H_1$ である．

Hybrid H_2**.** H_1 において，チャレンジビットに依存するコミットメントを，$b = 0$ 側の編集系列に対応するものから $b = 1$ 側の編集系列に対応するものへ順に置換する．各置換では，対象コミットメントに依存する Merkle root，Merkle 認証パス，履歴エントリ，および署名対象メッセージを整合するように再計算する．PC-NIZK 証明および CR-NIZK 証明については，変更後の公開ステートメントに対するシミュレートされた証明を生成する．

異なる非公開値に対する各コミットメントの置換は，コミットメント方式の秘匿性により計算量的に識別不能である．Merkle root や認証パスはコミットメントから効率的に計算される値であり，作成者署名および編集者署名も対応する公開メッセージに対して同一の署名アルゴリズムにより生成される．したがって，識別不能なコミットメントに対するこれらの効率的な後処理によっても識別不能性は保存される．よって $H_1 \approx_c H_2$ である．

すべての置換後，H_2 は $b = 1$ 側のコミットメント，Merkle 情報，および署名を持ち，PC-NIZK と CR-NIZK のみがシミュレートされた証明となっている分布である．なお，最終コンテンツ M_T はプライバシー試行の条件により両側で同一であるため，現在コンテンツそのものからチャレンジビットを識別することはできない．

Hybrid H_3**.** H_2 中の各 PC-NIZK のシミュレートされた証明および CR-NIZK のシミュレートされた証明を，$b = 1$ 側の実際のウィットネスから生成した正規証明へ順に置換する．同時に，必要に応じて公開パラメータも正規の setup から生成された分布へ戻す．両 NIZK のゼロ知識性より，$H_2 \approx_c H_3$ である．H_3 は $\mathsf{Exp}^{\mathsf{priv}-1}$ そのものである．

したがって，

$$\mathsf{Exp}^{\mathsf{priv}-0} \approx_c H_1 \approx_c H_2 \approx_c \mathsf{Exp}^{\mathsf{priv}-1},$$

であり，任意の PPT 攻撃者 $\mathcal{A}$ について $\mathsf{Adv}^{\mathsf{priv}}_{\text{CAPC-SS},\mathcal{A}}(\lambda)$ は無視できる．よって CAPC-SS は編集前情報の秘匿性を満たす． □

以上の補題 2-6 より，定理 1 が成り立つ．

A.6 保証範囲と運用上の前提

CAPC-SS は，受理された現在コンテンツと認証情報について，作成者が認証した初期状態から現在状態までルートが連続し，各履歴エントリが正当な編集者によって生成され，対応する編集が作成者のポリシーを満たすことを保証する．一方，以下の事項は CAPC-SS 単独では保証しない．

最新性．同一の有効な状態 (Σ, open) に対して複数の編集者が Sanitize を実行した場合，同じ作成者アンカーから分岐する複数の有効な履歴が生成され得る．CAPC-SS は各履歴の正当性を個別に検証するが，それらを一つの順序へ統合することや，提示された状態が最新であることまでは保証しない．履歴の一意性や最新性が必要な場合には，追記専用ログ，時刻認証，版管理等の外部機構と組み合わせる．

編集前情報の管理．open は，各セグメントの現在値とコミットメントの開示情報を含むため，編集を実行する主体間で安全に受け渡す必要がある．本稿の編集前情報の秘匿性は外部の検証者に対する性質であり，open を受け取る編集者や，その管理環境が侵害された場合の情報漏えいは対象外である．

外部編集．CAPC-SS は，編集アプリケーションによる操作内容の申告をそのまま信頼するのではなく，編集前後のセグメント値が Pol を満たすことを PC-NIZK によって検証する．また，現在アセットから segid に従って導出したセグメント列と最終ルートとの対応を CR-NIZK によって確認する．ただし，segid が示すセグメント化・正規化規則や Pol の意味が利用目的に適しているかは，作成者および利用システムが定める運用上の事項である．

公開情報．編集前情報の秘匿性が保護するのは，コミットメントに格納された編集前値とその開示情報である．一方，履歴には，編集者識別子 j，編集対象位置 i，公開編集後値 m'_i，入出力ルート，および履歴長 t が含まれる．したがって，どの編集者がどの位置を編集したか，何回の編集が行われたかといった履歴メタデータは秘匿されない．これらの情報まで秘匿することは，本稿の対象外である．

ポリシーおよび編集者鍵の更新．Pol，$\mathcal{PK}$，segid，および ppid は作成者署名の対象であり，一つの履歴系列では固定される．ポリシーの変更や編集者鍵の失効・更新が必要な場合には，現在コンテンツを新たな初期コンテンツとして更新後の設定で Sign を実行するか，外部の版管理・鍵失効機構と組み合わせる必要がある．

© 2026 Information Processing Society of Japan